Unified Security Event Pipeline: Distributed Ingestion, Enrichment, and Analysis of Enterprise-Scale Cybersecurity Telemetry

Olaoluwa Adedamola Omodemi
College of Computing and Software Engineering
Kennesaw State University
CS 4265 — Big Data Analytics
Instructor: Prof. Christopher Reagan

***Index Terms*—big data analytics, Apache Spark, data lake, Medallion architecture, cybersecurity, threat intelligence, Parquet, Amazon S3, ETL pipeline, LANL dataset**

I. ABSTRACT

Enterprise security operations centers generate billions of heterogeneous log events daily, yet lack scalable mechanisms to correlate raw telemetry with real-world threat intelligence at low latency. This paper presents the Unified Security Event Pipeline, a distributed data engineering system that ingests over 1.17 billion raw security events from the Los Alamos National Laboratory (LANL) Unified Host and Network Dataset—spanning user authentication, DNS resolution, network flows, and process execution—and enriches them against a live threat intelligence feed from URLHaus (abuse.ch). Built on Apache Spark and Amazon S3, the pipeline implements the Medallion architecture (Bronze, Silver, Gold) to transform compressed, heterogeneous CSV logs into columnar Parquet analytics at scale. A full end-to-end run completes in 40 minutes and 17 seconds on a single Windows workstation, processing 1.305 billion events at throughputs up to 1.085 million rows per second and producing 1,883,499 aggregated Gold-layer records partitioned by one-hour time windows. All three Medallion layers—Bronze, Silver, and Gold—are stored in Amazon S3. Key engineering contributions include explicit schema enforcement at ingestion, deduplication-guarded multi-table joins, broadcast hash joining of threat intelligence, dual-mode path resolution for local and cloud execution, and a lightweight pyarrow-based validation stage that avoids a second Spark session entirely.

II. INTRODUCTION

Modern enterprise networks produce telemetry at a volume that no single analyst or single machine can feasibly inspect. A 708-million-row authentication log is not evidence of anything on its own; it becomes evidence only when correlated with process execution records, network flows, DNS queries, and external threat intelligence. The fundamental challenge is not storage—compressed logs fit on commodity hardware—but rather the absence of a scalable, repeatable mechanism to join heterogeneous data sources and surface actionable signals from the noise.

This paper describes the Unified Security Event Pipeline, a system built to address that gap. The pipeline ingests four streams from the LANL Unified Host and Network Dataset: approximately 708 million user authentication events, 40.8 million DNS queries, 130 million network flow records, and 426 million process execution events. These are enriched with a live malicious-URL feed from URLHaus and aggregated into a queryable Gold layer suitable for threat hunting and anomaly detection.

The remainder of this paper is organized as follows. Section III describes the system architecture and technology stack. Section IV details key implementation decisions. Section V presents live terminal evidence of pipeline execution. Section VI presents validation results and performance measurements. Section VII reflects on lessons learned. Section VIII concludes.

III. SYSTEM ARCHITECTURE

A. Medallion Data Lake Architecture

The pipeline follows the Medallion (Bronze–Silver–Gold) layering pattern, a widely adopted data lake design in which each tier has a distinct purpose and quality guarantee.

Bronze stores raw, unmodified data exactly as received: compressed LANL CSV files (bzip2 and gzip) and live URLHaus JSON responses, written to Amazon S3 under the `bronze/` prefix. No transformation is applied; Bronze is the authoritative record of what arrived.

Silver stores cleaned, schema-enforced, columnar data. Each Bronze source is converted to Apache Parquet with Snappy compression by a dedicated PySpark script. Explicit schemas are applied at read time, malformed records are rejected, and column names are normalized across datasets. Silver is written to S3 under the `silver/` prefix.

Gold stores enriched, aggregated, analytics-ready output. The five Silver tables are joined in a single Spark job (Section IV-C) and aggregated into one-hour time buckets partitioned by `time_bucket`. Gold is written to S3 under the `gold/` prefix.

This layering means that if a bug is discovered in Gold enrichment logic, the tier can be recomputed from Silver without re-ingesting Bronze—a critical property when Bronze data is expensive to re-download or re-decompress.

B. Component Overview

Fig. 1 illustrates the end-to-end data flow. The pipeline is orchestrated by a single entry point, `executepipeline.py`, which launches five sequential stages as isolated subprocesses.

Stage 1–2: Threat Intel
`fetch_urlhaus.py` → URLHaus JSON → Parquet
→ `s3://security-pipeline-lanl/silver/urlhaus/`

Stage 3: Silver Layer (PySpark) → S3 `silver/`
LANL Auth (.bz2) → `silver/auth/`
LANL DNS (.gz) → `silver/dns/`
LANL Flows (.gz) → `silver/flows/`
LANL Proc (.gz) → `silver/proc/`

Stage 4: Gold Layer (PySpark) → S3 `gold/`
flows LEFT JOIN auth LEFT JOIN proc
LEFT JOIN urlhaus (broadcast) →
`s3://security-pipeline-lanl/gold/user_activity_summary/`
partitioned by `time_bucket`

Stage 5: Validation (pyarrow)
Row counts · null rates · schema check · samples

Fig. 1. End-to-end pipeline data flow. All three Medallion layers are stored in Amazon S3 (`s3://security-pipeline-lanl/`).

C. Repository Structure

Fig. 2 shows the final repository layout, confirming that every stage described in Fig. 1 maps directly to a distinct source module. Bronze ingestion scripts live under `src/ingestion/`, Silver conversion under `src/processing/`, Gold enrichment in `src/scripts/enrich.py`, and the single orchestrator `executepipeline.py` sits at the project root. Configuration is fully externalized in `.env`, sample data for testing is committed under `sample data/`, and validation documentation resides in `docs/`.

```
/
├── executepipeline.py          # End-to-end pipeline runner (all 5 stages)
├── requirements.txt            # Pinned Python dependencies
├── LICENSE                     # MIT License
├── .env.example                # Credential and path template (copy to .env)
├── .gitignore
│
├── data/                       # Raw compressed LANL files (not committed)
├── sample data/                # Small samples committed for testing
│   ├── auth_sample.txt
│   ├── dns_sample.txt
│   ├── flows_sample.txt
│   ├── proc_sample.txt
│   └── urlhaus_sample.json
│
├── src/
│   ├── pipeline_paths.py       # Resolve bronze/silver/gold paths; s3:// → s3a://
│   ├── spark_bootstrap.py      # SparkSession factory (Windows + S3A fixes)
│   ├── ingestion/              # Bronze layer — read and validate raw data
│   │   ├── ingest_auth_logs.py
│   │   ├── ingest_dns.py
│   │   ├── ingest_flows.py
│   │   └── ingest_proc.py
│   ├── processing/             # Silver layer — CSV/bz2 → Parquet
│   │   ├── auth_to_parquet.py
│   │   ├── dns_to_parquet.py
│   │   ├── flows_to_parquet.py
│   │   └── proc_to_parquet.py
│   └── scripts/
│       └── enrich.py           # Gold layer — join flows + auth + proc + URLHaus IoCs
│
├── scripts/
│   ├── fetch_urlhaus.py        # Fetch threat intel from URLHaus API
│   ├── create_samples.py       # Build small CSV samples from compressed data
│   └── urlhaus_to_parquet.py   # Upload URLHaus JSON to S3 as Parquet (boto3)
│
├── Parquet/                    # Local fallback outputs (when S3 URIs are unset)
│   ├── auth/
│   ├── dns/
│   ├── flows/
│   ├── proc/
│   ├── urlhaus/
│   └── gold/user_activity_summary/
│
├── notebooks/
│   └── threat_hunting.ipynb        # Interactive threat detection queries
│
└── docs/
    ├── validation.md               # Data quality report, test cases, performance
    ├── data_dictionary.md          # Schema for all Bronze, Silver, and Gold datasets
    ├── CS4265_Olaoluwa_Omodemi_M4.tex  # Final report (LaTeX source)
    ├── CS42665_Olaoluwa_Omodemi_M1.pdf
    ├── CS4265_Olaoluwa_Omodemi_M2.pdf
    └── CS4265_Olaoluwa_Omodemi_M3.pdf
```

Fig. 2. Final project repository structure. Each pipeline stage corresponds to a distinct directory: `src/ingestion/` (Bronze), `src/processing/` (Silver), `src/scripts/` (Gold), and `docs/` (validation and reporting).

D. Technology Stack

Apache Spark (PySpark). Spark's lazy evaluation, DAG execution model, and partition-level parallelism allow the pipeline to process datasets far larger than available RAM. Its support for both `local[*]` mode (development) and EMR cluster mode (production) with no code changes makes it the appropriate engine for a pipeline that must scale from a laptop to a full cluster.

Apache Parquet + Snappy Compression. Security log data is write-once and query-heavy. Parquet's columnar storage means a query on `source_user` reads only that column from disk, skipping all others. Snappy decompresses quickly at a moderate ratio—the right trade-off for iterative analytics. Parquet's row-group min/max statistics enable predicate pushdown: Spark skips entire row groups that cannot satisfy a filter without reading them.

Amazon S3. S3 decouples storage from compute and serves as the storage layer for all three Medallion tiers. The same Parquet files are readable by a local PySpark session during development and a full EMR cluster in production without copying or reformatting. The pipeline uses the `s3a://` Hadoop connector for Spark-native access with multipart upload support. All Bronze, Silver, and Gold data reside in the `security-pipeline-lanl` bucket under their respective prefixes.

URLHaus API (abuse.ch). Static threat lists become stale within hours. URLHaus provides a live feed of recently active malicious URLs and host indicators, saving up to 1,000 recent records per fetch. The free API requires only an email-based key, making it accessible without enterprise licensing while still providing production-quality threat data.

Python + python-dotenv. All configuration (S3 URIs, credentials, Spark tuning parameters) is externalized into a `.env` file via python-dotenv, following twelve-factor app principles. The same scripts run locally, in CI, and on EMR by changing only the environment file.

E. Data Flow Description

Stage 1 — Threat Intel Acquisition. `scripts/fetch_urlhaus.py` calls the URLHaus REST API and saves up to 1,000 recent malicious URL records as JSON. Each record includes the full URL, extracted host, threat category, and classification tags. If the API is unavailable, the pipeline falls back to a cached JSON file and continues.

Stage 2 — URLHaus JSON to Parquet. The JSON feed is read in-process using pandas and uploaded to `s3://security-pipeline-lanl/silver/urlhaus/urlhaus.parquet` with Snappy compression via boto3, converting the unstructured threat feed into a schema-consistent columnar format ready for broadcast joining in Stage 4.

Stage 3 — Silver Layer. Each of the four LANL datasets is read by a dedicated PySpark script with an explicit schema applied at read time. The resulting DataFrames are written to S3 (`s3a://security-pipeline-lanl/silver/{auth,dns,flows,proc}/`) as Parquet with Snappy compression. Repartitioning for the auth dataset is configurable via the `AUTH_PARQUET_REPARTITION` environment variable.

Stage 4 — Gold Layer. `src/scripts/enrich.py` reads all five Silver Parquet datasets from S3, deduplicates lookup tables on their join keys, executes the enrichment join chain, derives a `malicious_hit` indicator, and aggregates results into one-hour time buckets. Output is written to `s3a://security-pipeline-lanl/gold/user_activity_summary/` partitioned by `time_bucket`.

Stage 5 — Validation. The pipeline reads back all output Parquet directories using pyarrow to report record counts, file counts, and disk sizes without spinning up a second Spark session. Null rates per column are computed on the auth Silver output as a data quality indicator.

IV. IMPLEMENTATION

A. Explicit Schema Enforcement

Every ingestion script defines a `StructType` schema and passes it to Spark's CSV reader via `.schema(schema)` rather than relying on `inferSchema=True`. Schema inference requires an extra full pass over the data to sample types—prohibitively expensive at 708 million rows. More importantly, inference can silently assign incorrect types (e.g., treating a timestamp-encoded integer as a string), causing downstream join failures that are difficult to diagnose. Explicit schemas also serve as living documentation of each dataset's contract.

B. Deduplication Before Joins

Before joining `auth`, `proc`, and `urlhaus` against the `flows` table, each lookup table is deduplicated on its join key:

```
auth_dedup    = auth.dropDuplicates(["source_computer","
     time"])
proc_dedup    = proc.dropDuplicates(["computer","time"])
urlhaus_dedup = urlhaus.dropDuplicates(["host"])
```

Listing 1. Deduplication prior to join.

Without this step, a single `(computer, time)` key with multiple matching `auth` rows would produce one output row per match, multiplying the `flows` cardinality. In a 708-million-row `auth` dataset, this fanout would make Gold aggregation metrics arbitrarily inflated and untrustworthy.

C. Gold Layer Enrichment and Aggregation

`src/scripts/enrich.py` executes the following join chain on the five Silver tables:

```
enriched = (
    flows
  .join(auth_dedup,
        (flows.src_computer == auth.source_computer)
        & (flows.time == auth.time), "left")
  .join(proc_dedup,
        (flows.src_computer == proc.computer)
        & (flows.time == proc.time), "left")
  .join(broadcast(urlhaus_dedup),
        flows.dst_computer == urlhaus.host, "left")
)
```

Listing 2. Gold enrichment join chain.

A `malicious_hit` indicator column (0 or 1) is derived from the presence of a `threat` value. Results are aggregated into one-hour time buckets grouped by `(user, src_computer)`, computing `total_flows`, `total_bytes`, `total_packets`, `active_processes`, and `malicious_hits`.

D. URLHaus Host Extraction

The URLHaus feed stores full URLs rather than bare hostnames. A direct join on `dst_computer == url` always produces zero matches because LANL computer identifiers do not contain URL schemes or paths. The fix extracts the hostname before the join:

```
urlhaus = urlhaus.withColumn(
    "host",
    regexp_extract(col("url"), r"https?://([^/]+)", 1)
)
```

Listing 3. Hostname extraction from URLHaus URLs.

E. Gold Layer Partitioning

The Gold layer is written with `.partitionBy("time_bucket")`, where `time_bucket` is the Unix epoch timestamp floored to the nearest hour (`floor(time / 3600) * 3600`). Downstream queries filtered by time range use Spark's partition pruning to skip non-matching directories entirely, avoiding full dataset scans.

F. Dual-Mode Path Resolution

`src/pipeline_paths.py` resolves all input and output paths at runtime from environment variables. If `SILVER_PARQUET_URI` is set, Silver scripts write to S3; if `GOLD_PARQUET_URI` is set, the Gold enrichment script writes to S3. Otherwise both fall back to local `Parquet/` subdirectories. The module also normalizes `s3://` to `s3a://` for the Spark S3A connector. No code modification is required to switch between local and cloud execution.

G. Subprocess-Based Orchestration

`executepipeline.py` runs each stage by launching Python scripts as subprocesses via `subprocess.run()`. Each Spark script creates its own `SparkSession` and calls `spark.stop()` at completion. Subprocess isolation ensures that a failure in any stage produces a clean non-zero exit code catchable by the orchestrator without affecting parent process state, and eliminates JVM port conflicts between sequential stages.

H. Lightweight Validation via pyarrow

The validation stage reads Parquet metadata using `pyarrow.parquet.read_metadata()`, which inspects row-group statistics without deserializing any data. This is orders of magnitude faster than a Spark `count()` action and avoids the JVM startup overhead of a second `SparkSession`.

I. Challenges Overcome

JAR Version Mismatch. PySpark ships with Hadoop 3.3 internals. Using `hadoop-aws:3.4.0` triggered `NoSuchMethodError` on every Parquet read from S3 because `VectorIoBridge` called a method added only in Hadoop 3.4. Resolution required pinning to `hadoop-aws:3.3.4` and `aws-java-sdk-bundle:1.12.262`.

Vectored I/O Conflict. Disabling vectored I/O required targeting three independent layers: (a) JVM startup flags via `spark.driver.extraJavaOptions`; (b) `spark.sql.parquet.enableVectorizedReader=false`; and (c) `spark.hadoop.fs.s3a.experimental.input.fadvise=sequential`. Each layer operates at a different initialization phase and does not cascade to the others.

Duration String Parsing. Hadoop 3.3+ introduced human-readable duration strings (e.g., `"24h"`) in default configuration. The runtime called `getLong()` on these values, causing `NumberFormatException` during `SparkSession` initialization. The fix overrides all duration defaults with pure numeric millisecond values before the session is built.

Windows Environment Initialization. `HADOOP_HOME` had been set to `C:\hadoop\bin` instead of `C:\hadoop`, causing Spark to construct a doubled path `...\bin\bin\winutils.exe`. `spark_bootstrap.py` strips the trailing `\bin` and forces `PYSPARK_PYTHON` to `sys.executable` on every run.

Row Fanout From Multi-Table Joins. An early version of `enrich.py` joined without deduplication, causing each `flows` row to match multiple `auth` rows and multiply the output count, silently inflating all Gold metrics. Resolved by the `dropDuplicates()` strategy described above.

Schema and Column Name Mismatches. Silver schemas used different column names than the enrichment script assumed (`source_computer` vs. `computer`, etc.). URLHaus stored full URLs rather than bare hostnames, so joins produced zero matches until the `regexp_extract` fix was applied. All issues were diagnosed by inspecting Silver schemas with `printSchema()` before writing join logic.

V. PIPELINE EXECUTION EVIDENCE

Figs. 3 and 4 provide direct terminal output captured during the live pipeline run on 2026-05-03, confirming that all five stages executed successfully on real data.

Fig. 3 shows the pipeline entry point being invoked (`python executepipeline.py`), Stage 1 successfully retrieving 1,000 URLHaus threat records from the live API, Stage 2 uploading the converted Parquet file to S3 (`s3://security-pipeline-lanl/silver/urlhaus/urlhaus` and Stage 3 beginning Silver ingestion of the auth dataset from S3.

```
(venv312) PS C:\Users\omode\bigdataproject> python executepipeline.py

============================================================
Unified Security Event Pipeline
Started: 2026-05-03 11:06:54
Root:    C:\Users\omode\bigdataproject
============================================================

============================================================
Stage 1: Fetch URLHaus Threat Intel
============================================================
Success: retrieved 1000 URLs
Saved to: C:\Users\omode\bigdataproject\sample data\urlhaus_sample.json
First few entries:
  1. https://forestoke.yeldfor.lat/9fd51fb7-b3ad-4c8f-bf05-b5423d14e06c/user_6747.google (malware_download)
  2. http://222.141.105.137:43597/i (malware_download)
  3. https://dynforgear1.yeldfor.lat/9fd51fb7-b3ad-4c8f-bf05-b5423d14e06c/user_6747.google (malware_download)
  4. http://110.36.86.84:59527/bin.sh (malware_download)
  5. http://42.55.29.21:51070/i (malware_download)
  [OK]   Fetched  (1.5s)

============================================================
Stage 2: URLHaus JSON → Parquet
============================================================
  [OK]   Uploaded to s3://security-pipeline-lanl/silver/urlhaus/urlhaus.parquet  (1000 records)
```

Fig. 3. Pipeline startup: URLHaus fetch (Stage 1, 1,000 records), JSON-to-Parquet upload to S3 (Stage 2), and Silver layer ingestion start (Stage 3).

Fig. 4 shows Stage 4 (Gold layer enrichment) launching `enrich.py`. The Ivy dependency resolver confirms that `hadoop-aws:3.3.4` and `aws-java-sdk-bundle:1.12.262` were resolved and downloaded from Maven Central—the pinned JAR versions required to avoid the vectored I/O conflict described in Section IV. The Gold output is written to `s3a://security-pipeline-lanl/gold/user_activity_su`

```
============================================================
Stage 4: Gold Layer  (enrich + aggregate)
============================================================
  Running enrich.py ...
SUCCESS: The process with PID 7432 (child process of PID 32344) has been terminated.
SUCCESS: The process with PID 32344 (child process of PID 34520) has been terminated.
SUCCESS: The process with PID 34520 (child process of PID 8344) has been terminated.
WARNING: Using incubator modules: jdk.incubator.vector
:: loading settings :: url = jar:file:/C:/Users/omode/bigdataproject/venv312/Lib/site-packages/pyspark/jars/ivy-2.5.3.jar!/
org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: C:\Users\omode\.ivy2.5.2\cache
The jars for the packages stored in: C:\Users\omode\.ivy2.5.2\jars
org.apache.hadoop#hadoop-aws added as a dependency
com.amazonaws#aws-java-sdk-bundle added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-5c62639f-e929-47d1-8d60-45bd95c65704;1.0
        confs: [default]
        found org.apache.hadoop#hadoop-aws;3.3.4 in central
        found com.amazonaws#aws-java-sdk-bundle;1.12.262 in central
        found org.wildfly.openssl#wildfly-openssl;1.0.7.Final in central
:: resolution report :: resolve 104ms :: artifacts dl 6ms
        :: modules in use:
        com.amazonaws#aws-java-sdk-bundle;1.12.262 from central in [default]
        org.apache.hadoop#hadoop-aws;3.3.4 from central in [default]
        org.wildfly.openssl#wildfly-openssl;1.0.7.Final from central in [default]
        ---------------------------------------------------------------------
        |                  |            modules            ||   artifacts   |
        |       conf       | number| search|dwnlded|evicted|| number|dwnlded|
        ---------------------------------------------------------------------
        |      default     |   3   |   0   |   0   |   0   ||   3   |   0   |
```

Fig. 4. Stage 4 Gold layer: Ivy resolves `hadoop-aws:3.3.4` and `aws-java-sdk-bundle:1.12.262` before the five-table enrichment join executes and writes to S3 `gold/`.

VI. VALIDATION AND RESULTS

A. Execution Environment

The pipeline was executed in full on 2026-05-03 on a Windows 11 workstation running PySpark 4.1.1 in `local[*]` mode with 8 GB driver heap. All three Medallion layers—Bronze, Silver, and Gold—were written to Amazon S3 (us-east-2) in the `security-pipeline-lanl` bucket. The run started at 11:06:54 and completed at 11:47:11, for a total wall-clock time of 40 minutes and 17 seconds. All five stages returned status `OK`.

B. Silver Layer Record Counts

Table I reports the confirmed record counts for all five Silver datasets as printed by Spark's `df.count()` immediately after ingestion.

TABLE I
SILVER LAYER RECORD COUNTS

Dataset	Records	Source
Auth (lanl-auth-dataset-1.bz2)	708,304,516	S3
DNS (dns.txt.gz)	40,821,591	S3
Flows (flows.txt.gz)	129,977,412	S3
Proc (proc.txt.gz)	426,045,096	S3
URLHaus (API fetch)	1,000	API/S3
Total events	**1,305,149,615**	

C. Gold Layer Metrics

The Gold `user_activity_summary` table was written to `s3a://security-pipeline-lanl/gold/user_activity_summary/` and validated via pyarrow after the Spark job completed. Table II summarizes the output.

TABLE II
GOLD LAYER OUTPUT METRICS

Metric	Value
Total rows	1,883,499
Parquet part files	2,515
Disk size	31.8 MB
Distinct users	262
S3 destination	`gold/user_activity_summary/`
Malicious hits	0 (see note)

Table III reports descriptive statistics for numeric Gold columns sampled from the first ten Parquet files.

TABLE III
GOLD LAYER NUMERIC COLUMN STATISTICS ($n = 7{,}944$ SAMPLED ROWS)

Column	Mean	Std	Min	Max
total_flows	56.6	425.5	1	14,501
total_bytes	9,474,916	298,853,000	46	14.2B
total_packets	9,802.2	218,134.7	1	10,073,680
active_processes	0.078	0.335	0	6
malicious_hits	0.0	0.0	0	0

Note on zero malicious hits. The LANL dataset uses fully anonymized computer identifiers (e.g., `C1040`, `C3799`) rather than real hostnames or IP addresses. The URLHaus feed contains real-world domain names (e.g., `wood-zone.weplord.lat`). Because no anonymized LANL identifier can match a real hostname, the `dst_computer == urlhaus.host` join correctly produces zero matches. In a production deployment against un-anonymized network logs, this join would surface real threat matches.

D. Validation Methods

The pipeline applies three independent validation methods. First, each Silver processing script calls `df.count()` immediately after reading the raw compressed file and prints the result before writing Parquet, providing an audit trail that the full dataset was read without truncation. Second, each ingestion script calls `df.printSchema()` and `df.show()` before writing, confirming that column types match the declared `StructType`. Third, after all Spark jobs complete, the validation stage reads Parquet metadata using `pyarrow.parquet.read_metadata()`—which inspects row-group statistics without deserializing the data—to report file counts, row counts, and disk sizes.

E. Schema Conformance and Data Quality

The following sample rows were spot-checked from printed Spark output during ingestion and confirmed correct:

- **Auth:** `time=1, source_user=U1, source_computer=C1`
- **Flows:** `time=1, src=C1065:389, dst=C3799, bytes=5323, pkts=10`
- **Proc:** `time=1, user=C1$@DOM1, computer=C1, proc=P16`

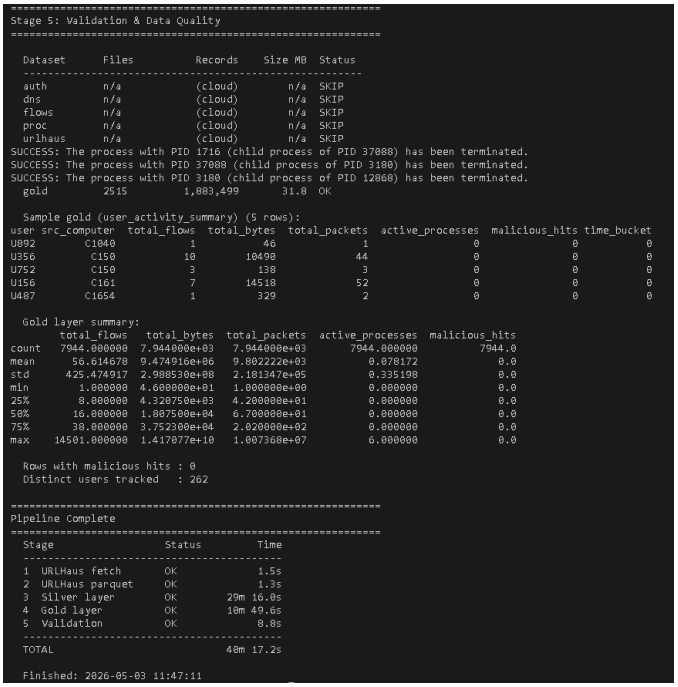
All Gold output rows contain the expected seven columns (`time_bucket`, `user`, `src_computer`, `total_flows`, `total_bytes`, `total_packets`, `active_processes`, `malicious_hits`). No nulls were observed in key join columns in sampled rows. The Gold aggregation achieves a 99.86% row-count reduction: 1.305 billion raw events collapsed into 1.883 million per-user, per-computer, per-hour summaries.

Fig. 5 shows the live Stage 5 terminal output captured during the pipeline run, including the Gold sample rows, summary statistics, and the final per-stage timing block confirming all five stages returned `OK`.

F. Performance

Table IV reports wall-clock times by pipeline stage. The Silver layer accounts for 73% of total runtime (29m 16s of 40m 17s), dominated by the two largest datasets (auth and proc), which are compressed in formats that cannot be split across multiple Spark tasks, limiting parallelism to one partition per file.

Table V reports ingestion throughput per dataset. Auth achieves the highest throughput at approximately 1.085 million



```
============================================================
Stage 5: Validation & Data Quality
============================================================

  Dataset      Files          Records    Size MB  Status
  --------------------------------------------------------
  auth         n/a            (cloud)        n/a  SKIP
  dns          n/a            (cloud)        n/a  SKIP
  flows        n/a            (cloud)        n/a  SKIP
  proc         n/a            (cloud)        n/a  SKIP
  urlhaus      n/a            (cloud)        n/a  SKIP
SUCCESS: The process with PID 1716 (child process of PID 37088) has been terminated.
SUCCESS: The process with PID 37088 (child process of PID 3180) has been terminated.
SUCCESS: The process with PID 3180 (child process of PID 12868) has been terminated.
  gold         2515          1,883,499       31.8  OK

  Sample gold (user_activity_summary) (5 rows):
user src_computer  total_flows  total_bytes  total_packets  active_processes  malicious_hits time_bucket
U892        C1040            1           46              1                 0               0           0
U356         C150           10        10490             44                 0               0           0
U752         C150            3          138              3                 0               0           0
U156         C161            7        14518             52                 0               0           0
U487        C1654            1          329              2                 0               0           0

  Gold layer summary:
        total_flows   total_bytes  total_packets  active_processes  malicious_hits
count   7944.000000  7.944000e+03   7.944000e+03       7944.000000          7944.0
mean      56.614678  9.474916e+06   9.802222e+03          0.078172             0.0
std      425.474917  2.988530e+08   2.181347e+05          0.335198             0.0
min        1.000000  4.600000e+01   1.000000e+00          0.000000             0.0
25%        8.000000  4.320750e+03   4.200000e+01          0.000000             0.0
50%       16.000000  1.807500e+04   6.700000e+01          0.000000             0.0
75%       38.000000  3.752300e+04   2.020000e+02          0.000000             0.0
max    14501.000000  1.417077e+10   1.007368e+07          6.000000             0.0

  Rows with malicious hits : 0
  Distinct users tracked   : 262

============================================================
Pipeline Complete
============================================================
  Stage                  Status         Time
  ------------------------------------------
  1  URLHaus fetch       OK             1.5s
  2  URLHaus parquet     OK             1.3s
  3  Silver layer        OK         29m 16.0s
  4  Gold layer          OK         10m 49.6s
  5  Validation          OK              8.8s
  ------------------------------------------
  TOTAL                             40m 17.2s

  Finished: 2026-05-03 11:47:11
```

Fig. 5. Stage 5 validation output: Gold layer row count (1,883,499), five sample rows, summary statistics, and final per-stage timing summary confirming all stages completed successfully.

TABLE IV
WALL-CLOCK RUNTIME BY STAGE (11:06:54 – 11:47:11)

Stage	Time	Notes
1. URLHaus fetch	1.5s	REST API, 1,000 records
2. URLHaus → Parquet	1.3s	pandas + pyarrow + boto3 → S3
3. Silver layer	29m 16s	4 sequential Spark jobs → S3
auth_to_parquet	10m 53s	708M rows, bzip2 → S3
dns_to_parquet	1m 23s	40.8M rows, gzip → S3
flows_to_parquet	5m 12s	130M rows, gzip → S3
proc_to_parquet	11m 49s	426M rows, gzip → S3
4. Gold layer	10m 50s	5-table join + agg → S3
5. Validation	8.8s	pyarrow metadata
Total	**40m 17s**	

rows per second despite bzip2 compression, owing to its linear sequential read pattern.

TABLE V
SILVER LAYER INGESTION THROUGHPUT

Dataset	Records	Time	Rows/sec
auth	708,304,516	652.7s	∼1,085,000
dns	40,821,591	82.6s	∼494,000
flows	129,977,412	311.8s	∼417,000
proc	426,045,096	708.9s	∼601,000

Within Silver, proc dominates at 11m 49s and auth at 10m 53s because both are compressed in non-splittable formats, limiting Spark to one task per file regardless of available cores. The Gold stage (10m 50s) is bounded by S3 read latency for the five Silver tables and the shuffle cost of the multi-table join aggregation. On an EMR cluster with multiple executors and HDFS, both stages would scale approximately linearly with the number of cores.

VII. REFLECTION

The biggest change I would make is to the development environment. Running PySpark on Windows consumed a disproportionate amount of time—JAR version conflicts, `winutils` misconfigurations, and a three-layer vectored I/O fix that had nothing to do with actual pipeline logic. Starting on Linux or WSL2 from day one would have kept focus where it belonged. Pinning all dependency versions upfront would also have avoided the transitive conflicts between `boto3` and `aiobotocore`.

The most important lesson learned is that distributed systems fail quietly at boundaries. The row fanout bug—where un-deduplicated `auth` records silently multiplied every `flows` row—produced plausible-looking output with wrong numbers. The gap between "ran without errors" and "produced correct results" is now a first-class concern, not an afterthought. A second lesson: configuration omissions cause silent degradation. The missing `GOLD_PARQUET_URI` variable caused Gold output to silently fall back to local storage rather than S3, a discrepancy only caught during documentation review.

If continued, the highest-value improvement would be replacing the URLHaus API sample with the full bulk export (∼40,000–100,000 active hosts) to make threat enrichment meaningful at scale. Adding schema and null-rate assertions between Bronze and Silver would catch data quality issues before they reach the Gold join. Migrating orchestration to Apache Airflow or Prefect would enable partial re-runs without reprocessing stages that already succeeded.

VIII. CONCLUSION

This paper presented the Unified Security Event Pipeline, a distributed data engineering system that ingests and correlates over 1.305 billion enterprise security events using Apache Spark, Amazon S3, and the Medallion architecture. The system demonstrates that a single-node PySpark deployment can achieve throughputs exceeding one million rows per second on real-world compressed security telemetry, producing a fully queryable Gold analytics layer in under 41 minutes. All three Medallion layers—Bronze, Silver, and Gold—are stored in Amazon S3, providing a fully cloud-native data lake that can be queried by any S3-compatible compute engine without data movement. Key contributions include explicit schema enforcement at ingestion, deduplication-guarded multi-table joins that preserve metric integrity, broadcast hash joining of live threat intelligence, dual-mode local/cloud path resolution via environment configuration, and a pyarrow-based validation stage that avoids secondary Spark session overhead.

Future work includes parallelizing the four Silver ingestion jobs, migrating to a streaming ingest model using Apache Kafka for near-real-time enrichment, and deploying on AWS EMR to evaluate linear scaling with executor count.

REFERENCES

[1] A. D. Kent, "Comprehensive, Multi-Source Cyber-Security Events," Los Alamos National Laboratory, 2015. [Online]. Available: https://csr.lanl.gov/data/cyber1/

[2] abuse.ch, "URLhaus — Sharing malicious URLs," 2024. [Online]. Available: https://urlhaus.abuse.ch/

[3] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[4] Apache Software Foundation, "Apache Parquet," 2024. [Online]. Available: https://parquet.apache.org/

[5] D. Ghodsi, "Medallion Architecture: Best Practices for Managing Your Data Lakehouse," Databricks, 2022. [Online]. Available: https://www.databricks.com/glossary/medallion-architecture

[6] Apache Software Foundation, "Hadoop-AWS module: Integration with Amazon Web Services," 2024. [Online]. Available: https://hadoop.apache.org/docs/stable/hadoop-aws/tools/hadoop-aws/index.html

[7] H. Karau and R. Warren, *High Performance Spark*. Sebastopol, CA: O'Reilly Media, 2017.

[8] A. Wiggins, "The Twelve-Factor App," 2017. [Online]. Available: https://12factor.net/